

ngspice Linear Solvers — KLU vs. Sparse 1.3

Behavior, defaults, and limitations across DC / AC / transient / noise

Ngspice + OpenVAF Enhancements project

July 2026

Contents

ngspice linear solvers — KLU vs. Sparse 1.3 (behavior, defaults, limitations)	1
TL;DR	1
Selecting and confirming the solver	1
Why KLU rejects noise and pole-zero	2
The one transient caveat — opamp741	2
How this was verified	2

ngspice linear solvers — KLU vs. Sparse 1.3 (behavior, defaults, limitations)

This build of ngspice-46 ships two direct linear solvers, and they are not fully interchangeable across analysis types. This note records exactly how they differ — which is the default, how to select and confirm each, and where KLU falls short — based on a direct read of the source and a solver-by-solver sweep of the whole [examples/](#) suite.

TL;DR

	KLU	Sparse 1.3
Availability in this build	Compiled in (SuiteSparse, statically linked — 44 klu_* symbols in the binary)	Always present (ngspice's own solver)
Default?	No	Yes — this build defaults to Sparse 1.3
How to select	<code>.option klu</code>	<code>.option sparse</code> (or just the default)
DC op / DC sweep	✓ correct	✓ correct
AC	✓ correct	✓ correct
Transient	✓ correct (one caveat below)	✓ correct
Noise (.noise)	✗ rejected — <code>noisean.c:78</code>	✓ correct
Pole-zero (.pz)	✗ rejected — <code>pzan.c:31</code>	✓ correct

Practical guidance: leave the default (Sparse 1.3) unless you have a specific reason to switch. Sparse 1.3 runs every analysis in the suite — DC, AC, transient, *and* noise/pole-zero. Reach for `.option klu` on large, sparse DC/AC problems where KLU's ordering and factorization are faster, but do not use it for noise or pole-zero (ngspice will error out), and expect it to be less robust on stiff transient edges (see opamp741 below).

Selecting and confirming the solver

The netlist option is a simple flag:

```
.option klu      * use KLU (SuiteSparse)
.option sparse   * use the legacy Sparse 1.3 (this is also the default)
```

Because unknown `.option` keywords are silently ignored by ngspice, "it ran without complaint" is *not* evidence the solver changed. To confirm which solver is actually active, use the interactive `option` command, which prints the live `CKTKluMODE`:

```
.control
run
option      * prints, among other things: "Matrix solver: KLU" or "Sparse 1.3"
.endc
```

On the committed bin/ binary this reports **Sparse 1.3** for a bare deck and for `.option sparse`, and **KLU** only when `.option klu` is given — confirming Sparse 1.3 is the default here.

Why KLU rejects noise and pole-zero

These are not silent wrong answers — ngspice refuses the analysis with an explicit diagnostic and aborts:

```
Error: Noise simulation is not (yet) supported with 'option KLU'.      (noisean.c:78)
Error: Pole/zero analysis is not (yet) supported with 'option KLU'.    (pzan.c:31)
```

The noise and pole-zero paths build and manipulate the small-signal matrix in a way that was only wired up for the Sparse 1.3 storage, not the KLU/CSC path. This is an upstream ngspice limitation, independent of the Verilog-A/OSDI work in this repository. The `.noise` and `.pz` engines themselves are correct — they simply require the Sparse 1.3 solver.

The one transient caveat — opamp741

The transistor-level [opamp741 example](#) (a μ A741 built from ~ 70 lines of Verilog-A BJT) is the single case in the suite where KLU and Sparse produce different transient results:

- Sparse 1.3: the large-signal slew test (`pulse(-5 5 ...)` into the follower) runs the full `tran 20n 80u` cleanly (~ 4058 points), slew rate ≈ 0.54 V/ μ s.
- KLU: the same run diverges at the slewing edge — `v(out)` overshoots past the -5 V rail to ≈ -6.16 V and ngspice aborts the timestep at $t \approx 2.03$ μ s (~ 133 points), so the characterization can't complete.

This is a convergence/timestep robustness difference on a stiff circuit, not a correctness bug in either solver: on the same circuit Sparse's pivoting survives the fast edge and KLU's does not. Smaller, better-conditioned transients (the other transient examples) run identically under both.

Note: this is exactly the class of hard-convergence problem that [Enhancement-111](#)'s globalized Newton targets — but that line search damps the DC operating-point Newton, not per-timepoint transient Newton iterations, so it does not address this particular transient divergence.

How this was verified

Every `verify_*.py` in [examples/](#) (101 machine-checkable scripts) was run twice — once with `.option klu` and once with `.option sparse` force-injected into every deck — and the pass/fail sets were diffed. Result:

- Sparse 1.3: 93/101. KLU: 82/101.
- All 11 differing examples pass under Sparse and fail under KLU, and every one is explained by the table above:
 - 9 run a `.noise` analysis $\rightarrow$ rejected under KLU (`analyses`, `finalstep`, `noisecorr`, `noisejw`, `noisetable`, `paramrange`, `paramsethsp`, `physcheck`, `tempphys`);
 - 1 is the `opamp741` transient divergence;
 - 1 (`groundcontrib`) is a flaky compile-only diagnostic, unrelated to the solver (passes on retry).
- A further 8 examples fail identically under both solvers — i.e. not solver-related at all: 7 are missing-Python-dependency issues (`numpy` / `matplotlib`) in the test environment, and 1 (`hierbranch`) is a pre-existing content check. These are orthogonal to the solver question.

Conclusion: for DC, AC, and transient the two solvers agree across the suite (the lone exception being the stiff `opamp741` transient under KLU), while noise and pole-zero are Sparse-1.3-only by ngspice design. Sparse 1.3, the default, covers everything.